COMS3011 Lab 1

AI Transcripts 05 August 2026

Purpose of this document

This document summarises how ChatGPT [GPT-5.6 Luna] and Claude [Claude Sonnet 5] were used during the development of the SDP Todo application lab. The AI was used as a development assistant rather than as a replacement for testing or understanding the submitted work. Suggestions were verified according to the lab's instructions and based on the actual performance of the application.

1 Constraints Given Up Front

The project brief specified that the application had to:

- use Next.js and SQLite;
- be local-first and intended for a single user;
- allow users to create, edit and archive tasks;
- store a title, description, due date and topic for each task;
- prevent permanent deletion of tasks;
- allow archived tasks to remain viewable;
- use the fixed statuses Todo, In-Progress and Complete;
- allow sorting by topic, status and due date;
- indicate overdue tasks without making overdue a status;
- persist all information after restarting the application;
- contain at least three tests exercising real behaviour;
- provide one documented command for running the tests; and
- contain documentation for Third-Party Code, Database Design and Running It.

The project was therefore approached incrementally. I explicitly stated that the require-

ments should be broken down into smaller milestones and that coding would continue only after these milestones had been completed.

2 Task 1 – Breaking the Assignment into Milestones

User request

The first step involved dividing the requirements of the lab brief into small milestones without coding. In this case, I preferred to implement the requirements one by one.

AI assistance

The assistant divided the project into phases:

1. Project setup
2. SQLite connection
3. Database schema
4. Basic task management
5. Task editing
6. Archiving
7. Sorting
8. Overdue detection
9. Testing
10. Documentation
11. Final review

Each phase was further broken down into smaller milestones, such as creating the project, connecting to SQLite, creating the task table, creating a task, showing tasks, modifying tasks, and archiving tasks.

Why this was useful

The use of milestones decreased the amount of work needed for each implementation stage and allowed for testing of one requirement before proceeding to another one. It helped create a consistent Git history, since the marking rubric required six commits.

3 Task 2 – Database Design

User request

I began by implementing the database and task functionality and used AI to reason about the database structure and how it should correspond with the application.

AI assistance

The database was designed around a `tasks` table containing fields corresponding to the required task information.

The final schema used snake_case database fields, including:

```
id
title
description
due_date
topic
status
archived_at
created_at
```

The status field was constrained to the three allowed values:

```
Todo
In Progress
Complete
```

One of the key design considerations was that the overdue attribute must not be a field in the database; it should be calculated using the due date and the status at the time of reading.

The archiving functionality had been incorporated into the task itself and not as a deletion of the record from the database.

Verification

Database design was constantly verified during the development of the API and testing process. Issues occurred in the future when there was an inconsistency between the fields of the application and those of the database. This led to further debugging.

4 Task 3 – Creating Tasks

User request

The requirement that a task should be created with its required information was implemented.

AI assistance

The assistant helped reason through the task creation flow:

1. The user enters the title.
2. The user enters the description.
3. The user selects or enters the due date.
4. The user provides the topic.
5. The task is inserted into SQLite.
6. The task becomes visible in the active task list.

The assistant also helped identify the relationship between the form, API route and database schema.

Verification

Task creation was later included in the automated tests. The task creation test checked real database/application behaviour rather than merely checking that a component rendered.

5 Task 4 – Displaying Tasks

User request

After creating tasks, I needed the application to display saved tasks.

AI assistance

The assistant helped reason about retrieving active tasks from SQLite and displaying the required task information, including:

- title;
- topic;
- due date; and
- status.

The active task list was designed to exclude archived tasks.

6 Task 5 – Editing Tasks

User request

I worked on allowing an existing task to be edited and ensuring that changes survived a page reload.

AI assistance

The assistant helped debug the edit flow, including the PATCH API route and the mapping between the form values and database fields.

The implementation was eventually changed to use the snake_case database names consistently, particularly `due_date` and `archived_at`.

Verification

I checked that an edited task could be read again after reloading the page. This was important because the assignment specifically requires edits to survive a page reload.

7 Task 6 – Archiving Instead of Deleting

User request

An implementation of the requirement that tasks could be archived but not permanently deleted was needed.

AI assistance

The assistant recommended representing the archived state on the existing task record rather than deleting the row.

The implementation used an archive timestamp:

```
archived_at
```

An archive operation set this value instead of deleting the task.

The active task list then filtered out archived tasks, while an archived-task view could still display them.

Verification

Archiving was tested as real behaviour. The archive test checked that the task was no longer part of the active list while its information remained stored and viewable.

8 Task 7 – Sorting

User request

The assignment required sorting the active task list by topic, status and due date.

AI assistance

The assistant helped identify the required sort fields and debug the dynamic sorting logic.

The valid sort fields were restricted to:

```
topic
status
due_date
```

This avoided allowing arbitrary database column names to be inserted into the SQL ordering expression.

Verification

The three required sorting modes were checked against the assignment's functional walk-through.

9 Task 8 – Overdue Logic

User request

I implemented the requirement that overdue tasks must be visibly indicated but must not become a fourth status.

AI assistance

The assistant helped define overdue as a derived condition.

The logic was based on the relationship between:

- the task's due date;
- the current date;
- whether the task is complete; and
- whether the task has been archived.

The key principle was that overdue should not be stored in SQLite and should not replace or add to the three fixed statuses.

Correction during development

I then noticed that overdue highlighting was not behaving as expected. The assistant helped examine the condition and clarify that an overdue indication should disappear when the task becomes complete, because the overdue state is derived from the current task state. The implementation was adjusted and tested again.

10 Task 9 – Status Constraint Error

Problem encountered

A significant debugging issue occurred when updating task status.

The SQLite schema used:

```
Todo
In Progress
Complete
```

while part of the TypeScript code used a different representation:

```
todo
in-progress
complete
```

This resulted in a SQLite `CHECK` constraint failure when an update attempted to insert a status value that did not match the database constraint.

AI assistance

The assistant identified the mismatch between the application's `TaskStatus` type and the SQLite `CHECK` constraint as the cause of the error.

The application and database representations were then made consistent.

11 Task 10 – Testing with Vitest

User request

The assignment required at least three tests exercising real behaviour, including at least one test covering archiving or overdue behaviour.

AI assistance

The assistant helped plan and debug tests for:

1. task creation;
2. task archiving; and

3. overdue-task logic.

The tests were designed to use database behaviour rather than being render-only tests.

The test suite eventually consisted of:

```
tests/archiving.test.ts
tests/overdue.test.ts
tests/task-creation.test.ts
```

with a total of eight tests.

Verification

The test suite was run and reached:

```
3 test files passed
8 tests passed
```

12 Task 11 – npm and Node.js Problems

Problem encountered

During development, i encountered npm installation problems on Windows, including an `EPERM` error involving files in `node_modules` and a native build failure involving `better-sqlite3` and `node-gyp`.

I also encountered issues related to Node.js versions and considered moving between Node.js 24 and Node.js 22.

AI assistance

The assistant helped diagnose these issues and explained that native SQLite packages such as `better-sqlite3` can require a compatible native build environment on Windows.

I was guided through possible Node.js and npm troubleshooting steps rather than treating the original installation failure as an application-code error.

13 Examples of AI Corrections and Author Decisions

Several development decisions demonstrate that AI suggestions were not accepted without verification.

1. A status-value mismatch between the application and SQLite schema caused a real `CHECK` constraint error. The mismatch was identified and corrected.

2. The overdue indication did not initially behave as intended. The overdue condition was reviewed and adjusted so that it was derived from the current task state.

3. Database field naming changed during development. The API and TypeScript code were updated to match the final snake_case database schema instead of leaving inconsistent names throughout the application.

4. npm and native SQLite installation errors were investigated using my actual Windows environment rather than assuming that the initial dependency setup would work.

These examples show that the AI was used interactively for debugging and decision support rather than only for generating a complete solution.

Conclusion

ChatGPT and Claude were used throughout the lab as planning, explanation, debugging and testing assistants. The most important use was breaking the lab into small requirements and then using AI to investigate concrete problems encountered during implementation.

AI Declaration — The preceding document was planned, reviewed, edited and generated with the assistance of Claude [Claude Sonnet 5].